



Writeup – Abort

Auteur du writeup : 5yn0r

CTF : JerseyCTF

Catégorie : Reverse Engineering

Contexte du challenge

The arcade cabinet's control node is failing, and maintenance access has been restricted. You've intercepted the executable powering the system, but only the compiled binary remains.

Reverse engineer the program, discover what input it expects, and restore the cabinet before the watchdog timer powers everything down.

Note: You will have limited time once connected.

```
nc abort.aws.jerseyctf.com 1337
```

Fichier fourni : `abort` (ELF 64-bit)

1. Reconnaissance initiale

Commandes exécutées :

```
file abort
checksec --file=abort
strings abort | grep -i flag
```

Résultats :

- ELF 64-bit, stripped
- Pas de canary, pas de PIE → vulnérabilité potentielle mais non exploitée ici
- `strings` montre `cat flagH` et `lag.txt` (indice d'un appel à `cat flag.txt`)

2. Analyse statique avec Ghidra

Le binaire étant **stripped**, il faut identifier la fonction principale.

En suivant l'entry point (`_start`), on trouve un appel à `__libc_start_main` avec l'adresse `0x401460` → c'est `main`.

Désassemblage de `main` (Ghidra)

```
void main(void)
{
    signal(0xe, FUN_004012ec);
    alarm(0x78);
    FUN_00401365();
}
```

La fonction `FUN_00401365` contient toute la logique.

3. Analyse de `FUN_00401365`

```
void FUN_00401365(void)
{
    int iVar1;
    undefined1 local_58 [64];
    undefined4 local_18;
    undefined4 local_14;
    undefined1 auStack_10 [8];
```

```

memset(local_58, 0, 0x50);
puts(...); // affichage des bannières
printf("Insert maintenance packet: ");
read(0, local_58, 0x50); // lecture de 80 bytes

iVar1 = FUN_00401216(local_18);
if (((iVar1 == 0x5a7eab95) &&
     (iVar1 = FUN_00401230(local_14), iVar1 == 0x6fa08e7
e)) &&
     (iVar1 = FUN_0040124a(auStack_10), iVar1 != 0))
{
    FUN_00401323(); // succès → affichage du flag
    return;
}
puts("\\n[ LINK FAILURE ]");
puts("[ CABINET REMAINS LOCKED ]");
}

```

Structure de l'input (80 bytes)

Offset	Taille	Variable	Rôle
0	64	local_58	padding (ignoré)
64	4	local_18	1er entier
68	4	local_14	2e entier
72	8	auStack_10	chaîne de 8 bytes

4. Reverse des fonctions de validation

a) FUN_00401216

```

int FUN_00401216(uint param_1)
{
    return (param_1 ^ 0x4b1d3f29) + 0x91a72c6e;
}

```

Condition : $(\text{input} \oplus 0x4b1d3f29) + 0x91a72c6e == 0x5a7eab95$

Résolution :

```
input ^ 0x4b1d3f29 = 0x5a7eab95 - 0x91a72c6e
                  = 0xc8d77f27
input = 0xc8d77f27 ^ 0x4b1d3f29 = 0x83ca400e
```

Valeur attendue (little-endian) : `0e 40 ca 83`

b) `FUN_00401230`

```
uint FUN_00401230(int param_1)
{
    return (param_1 + 0x94252611) ^ 0x6f6f6f6f;
}
```

Condition : `(input + 0x94252611) ^ 0x6f6f6f6f == 0x6fa08e7e`

Résolution :

```
input + 0x94252611 = 0x6fa08e7e ^ 0x6f6f6f6f = 0x00cfe111
input = 0x00cfe111 - 0x94252611 = 0x6caabb00 (mod 2^32)
```

Valeur attendue (little-endian) : `00 bb aa 6c`

c) `FUN_0040124a`

```
bool FUN_0040124a(byte *param_1)
{
    byte local_12[6] = {0x3d, 0x2e, 0x3f, 0x3d, 0x38, 0x39};
    for (int i = 0; i <= 5; i++) {
        if ((param_1[i] ^ 0x5c) != local_12[i])
            return false;
    }
    return param_1[6] == '\\0';
}
```

On résout :

```
param_1[i] = local_12[i] ^ 0x5c
```

- `0x3d ^ 0x5c = 0x61` → `'a'`

- `0x2e ^ 0x5c = 0x72` → `'r'`
- `0x3f ^ 0x5c = 0x63` → `'c'`
- `0x3d ^ 0x5c = 0x61` → `'a'`
- `0x38 ^ 0x5c = 0x64` → `'d'`
- `0x39 ^ 0x5c = 0x65` → `'e'`

Chaîne attendue (8 bytes) : `"arcade\\x00\\x00"`

5. Construction du payload

```
import sys

payload = bytearray(80) # 64 premiers bytes à 0

# offset 64 : 4 bytes
payload[0x40:0x44] = bytes([0x0e, 0x40, 0xca, 0x83])

# offset 68 : 4 bytes
payload[0x44:0x48] = bytes([0x00, 0xbb, 0xaa, 0x6c])

# offset 72 : 8 bytes
payload[0x48:0x4e] = b"arcade"
payload[0x4e] = 0
payload[0x4f] = 0

sys.stdout.buffer.write(payload)
```

6. Validation locale et flag

Test local

```
python3 exploit.py | ./abort
```

Sortie :

```
[ SYSTEM OVERRIDE ACCEPTED ]  
[ ACCESS LEVEL: ARCADE OPS ]  
[ DUMPING VAULT CONTENTS ]  
cat: flag.txt: Aucun fichier ou dossier de ce nom
```

✓ La validation est correcte (flag absent localement).

Connexion au serveur

```
python3 exploit.py | nc abort.aws.jerseyctf.com 1337
```

Flag obtenu :

```
jctf{$UccES5Fully_ab0rt3D!_c0nGRATUl@t!0ns}
```

Conclusion

Ce challenge était un **reverse engineering classique** avec :

- Binaire stripped
- Validation par étapes (deux entiers + une chaîne)
- Analyse statique via Ghidra
- Reconstruction du payload exact

La difficulté principale était de comprendre l'organisation mémoire des 80 bytes lus et de retrouver les constantes exactes des fonctions de validation.